Task: Develop a Simple CRUD REST API with Role-Based Access Control using JWT Authentication.

1. Authentication

- Implement JWT-based authentication.

- A user must login and receive a JWT token.

- All protected APIs must require a valid JWT token.

2. User Roles

The system must support three roles:

- User

- Checker

- Authorizer

3. User Role Permissions

- The User can perform CRUD operations on a resource (e.g., Request, Document, or Application).

- When a user creates data, the initial status should be 'Pending'.

4. Checker Role Permissions

- The Checker can view all pending records created by Users.

- The Checker can Approve or Reject records.

- If approved, the status becomes 'Checked'.

5. Authorizer Role Permissions

- The Authorizer can view only records that were approved by the Checker (status = Checked).

- The Authorizer can Approve (Authorize) or Reject them.

6. Workflow Logic

User → Create Data → Status: Pending

Checker → Reviews Data

If Approved → Status: Checked

If Rejected → Not visible to Authorizer

Authorizer → Can only see records with status = Checked

If Approved → Status: Authorized

7. Role-Based API Access

API endpoints must be protected using role-based middleware.

Example structure:

/api/user/* → accessible only by User

/api/checker/* → accessible only by Checker

/api/authorizer/* → accessible only by Authorizer

8. Role-Based Response Endpoint

Create an endpoint: GET /api/dashboard

The response must return a message based on the authenticated role:

User → 'This is User Page'

Checker → 'This is Checker Page'

Authorizer → 'This is Authorizer Page'

9. Expected Deliverables

- REST API implementation

- JWT authentication

- Role-based middleware

- Database schema

- README explaining how to run the project

Bonus (Optional)

- Refresh tokens

- Pagination for records

- API documentation using Swagger/OpenAPI

- Proper error handling and validation